

PORTADA

Análisis y Diseño de Sistemas II

Semana 8 — SOLID: Open/Closed (OCP) y Liskov Substitution (LSP)

Universidad de Antioquía · Ingeniería de Sistemas



APERTURA

**Cada vez que llega un caso nuevo, alguien edita
el mismo switch — y algo que ya funcionaba se rompe**

TRANSICIÓN

Dos principios más para cerrar la Unidad 4

El martes vimos **SRP**: una clase, una razón para cambiar. Hoy vemos los dos principios que le siguen en el acrónimo SOLID: **Open/Closed (O)**, sobre cómo agregar comportamiento sin arriesgar lo que ya funciona, y **Liskov Substitution (L)**, sobre cuándo una subclase puede reemplazar a su superclase con seguridad.

CONTRAEJEMPLO

El switch que crece con cada tipo nuevo

```
class CalculadoraDeDescuento:
    def calcular(self, tipo_cliente, monto):
        if tipo_cliente == "regular":
            return monto * 0.0
        elif tipo_cliente == "frecuente":
            return monto * 0.05
        elif tipo_cliente == "vip":
            return monto * 0.10
        # cada cliente nuevo = una rama más aquí
```

Agregar un tipo de cliente "**corporativo**" exige abrir esta clase, editar un método que ya estaba probado y funcionando, y volver a probar todo el `if/elif` completo — no solo el caso nuevo.

CONCEPTO

El problema: tocar código que ya funciona

Cada vez que se modifica un método que ya estaba en producción y ya tenía pruebas pasando, se corre el riesgo de introducir una **regresión**: romper un caso que antes funcionaba, al intentar agregar uno nuevo.

El `if/elif` del ejemplo anterior mezcla en un mismo bloque de código la lógica de **todos** los tipos de cliente, presentes y futuros — cualquier cambio, por pequeño que sea, pasa por el mismo punto de falla único.

CONCEPTO

Definición formal de OCP

Open/Closed Principle: las entidades de software (clases, módulos, funciones) deben estar **abiertas a extensión** pero **cerradas a modificación**.

- **Abierta a extensión:** se puede agregar comportamiento nuevo cuando el sistema lo necesite.
- **Cerrada a modificación:** agregar ese comportamiento nuevo no debería requerir editar el código fuente que ya existe, probado y en producción.

El principio lo formuló originalmente Bertrand Meyer en 1988 y Robert C. Martin lo incorporó como la "O" de SOLID.

CONCEPTO

La solución: polimorfismo en vez de condicionales

La semana pasada vimos que una **interfaz** es un contrato que varias clases pueden implementar, cada una con su propio comportamiento, y que el **polimorfismo de subtipo** permite tratarlas todas de manera uniforme. Esa es exactamente la herramienta que resuelve OCP.

```
interface EstrategiaDeDescuento:
    def calcular(self, monto): ...

class DescuentoRegular implements EstrategiaDeDescuento:
    def calcular(self, monto): return monto * 0.0

class DescuentoFrecuente implements EstrategiaDeDescuento:
    def calcular(self, monto): return monto * 0.05
```

CONCEPTO

Agregar un caso nuevo, sin tocar el viejo

```
class DescuentoVip implements EstrategiaDeDescuento:
    def calcular(self, monto): return monto * 0.10

class DescuentoCorporativo implements EstrategiaDeDescuento:
    def calcular(self, monto): return monto * 0.15 # clase NUEVA

def aplicar_descuento(estrategia: EstrategiaDeDescuento, monto):
    return estrategia.calcular(monto) # este código nunca cambia
```

DescuentoCorporativo se agrega como una **clase nueva**. aplicar_descuento() y todas las estrategias anteriores **quedan intactas** — literalmente cerradas a modificación, abiertas a extensión.

CONCEPTO

Por qué importa: el costo de re-probar

- **Menos riesgo de regresión:** el código que calculaba el descuento "vip" nunca se vuelve a tocar al agregar "corporativo".
- **Pruebas más baratas:** solo hay que escribir y correr pruebas nuevas para `DescuentoCorporativo`, no repetir toda la suite del método gigante.
- **Despliegue más seguro:** un cambio aislado en una clase nueva es más fácil de revisar en un pull request que un diff dentro de un método de 40 líneas.

TENSIÓN CONCEPTUAL

OCP no significa "prever todo desde el día uno"

Un error común es leer OCP como licencia para **sobre-diseñar**: crear interfaces y jerarquías de estrategias "por si acaso" se necesitan en el futuro, sin que exista todavía una variación real. Eso agrega complejidad e indirección que nadie usa.

OCP se aplica cuando el punto de variación ya se conoce o ya se repitió —como los tipos de descuento, que cambian cada trimestre—, no de forma especulativa sobre un caso que quizás nunca llegue. Empezar simple (incluso con un `if`) y refactorizar hacia OCP cuando aparece el segundo o tercer caso real suele ser mejor que anticiparlo todo.



INTERACCIÓN

¿Aplicar OCP ahora o esperar?

En el chat, para cada escenario decidan si conviene introducir polimorfismo ya (OCP) o esperar a que aparezca un segundo caso real:

1. Un sistema de facturación que hoy solo calcula IVA, y no hay planes de otro impuesto.
2. Un validador de formularios que ya tiene tres tipos de campo y el negocio anuncia dos más el próximo mes.
3. Un método que exporta reportes solo a PDF, "por si algún día" piden Excel.

5 minutos · respondan en el chat

TRANSICIÓN

La extensión con polimorfismo tiene una condición oculta

La solución de OCP funciona porque `aplicar_descuento()` confía en que **cualquier** implementación de `EstrategiaDeDescuento` se va a comportar de forma consistente. Esa confianza no es automática: es exactamente lo que garantiza el tercer principio de hoy.

CONCEPTO

Definición formal de LSP

Liskov Substitution Principle (Barbara Liskov, 1987): los objetos de una subclase deben poder **sustituir** a los objetos de su superclase en cualquier parte del programa, **sin alterar la corrección** de ese programa.

En otras palabras: si B es subclase de A , cualquier código escrito para trabajar con A debe seguir funcionando correctamente si en su lugar recibe un objeto de tipo B — sin que el código que lo usa necesite saberlo ni adaptarse.

CONTRA EJEMPLO

Cuadrado–Rectángulo: la violación clásica de LSP

```
class Rectangulo:
    def set_ancho(self, a): self.ancho = a
    def set_alto(self, h): self.alto = h

class Cuadrado(Rectangulo):
    def set_ancho(self, a):
        self.ancho = a; self.alto = a
    def set_alto(self, h):
        self.ancho = h; self.alto = h

def probar(r: Rectangulo):
    r.set_ancho(5); r.set_alto(4)
    assert r.ancho * r.alto == 20    # falla si r es Cuadrado
```

La semana pasada vimos este código sin nombrar el problema. Ahora sí: `Cuadrado` no puede **sustituir** a `Rectangulo` en `probar()` sin romper una suposición válida — es el ejemplo canónico de violación de LSP.

CONCEPTO

Señales de violación de LSP

Señal	Qué implica
Excepciones nuevas	La subclase lanza una excepción en un método donde la superclase nunca lanzaba nada
Precondiciones más fuertes	La subclase exige condiciones de entrada más estrictas que las que la superclase aceptaba
Postcondiciones más débiles	La subclase entrega un resultado que cumple menos garantías que las que la superclase prometía
Efecto secundario inesperado	Un método que en la superclase no modificaba cierto estado, en la subclase sí lo hace (como <code>set_ancho</code> alterando <code>alto</code> en <code>Cuadrado</code>)

CONCEPTO

Precondición y postcondición, con código

Precondición: lo que un método exige para poder ejecutarse. **Postcondición:** lo que un método garantiza al terminar. Una subclase viola LSP si es más exigente al entrar o cumple menos al salir que su superclase.

```
class RepositorioDeArchivos:
    def guardar(self, archivo):
        ... # acepta cualquier tamaño de archivo

class RepositorioDeArchivosEnCache(RepositorioDeArchivos):
    def guardar(self, archivo):
        if archivo.tamano > 1024:
            raise ValueError("archivo muy grande") # precondición NUEVA
        ...
```

Cualquier código que ya llamaba `guardar()` confiando en la superclase puede romperse al recibir la subclase — precondición fortalecida sin aviso.

CONCEPTO

LSP es lo que hace segura a la herencia

La semana pasada listamos cuándo **no** usar herencia, y uno de los criterios era: "cuando heredar rompe el comportamiento esperado de la superclase, la subclase debería poder sustituirla sin sorpresas". Eso era LSP dicho en palabras — hoy queda con nombre y reglas verificables.

LSP no prohíbe la herencia: le pone una condición. Una jerarquía de herencia es segura de usar exactamente cuando toda subclase cumple LSP frente a su superclase — si no lo cumple, la relación "es-un" que parecía obvia en el dominio no debería modelarse como herencia.

CONCEPTO · LO QUE VIENE

OCP y LSP se necesitan mutuamente

OCP propone extender el sistema agregando subclases o implementaciones nuevas, sin tocar el código existente. Eso solo es seguro si esas subclases **cumplen LSP** frente a la superclase o interfaz que extienden — si no la cumplen, "extender" en realidad significa "romper" en algún punto del sistema.

Los cinco principios SOLID se apoyan entre sí: SRP ordena las responsabilidades, OCP dice cómo crecer sin arriesgar lo existente, y LSP garantiza que esa forma de crecer —agregando subtipos— sea segura.

SÍNTESIS

Ideas clave de hoy

1. **OCP**: abierto a extensión, cerrado a modificación — agregar comportamiento sin tocar código ya probado.
2. El polimorfismo/interfaces (semana 7) es la herramienta que hace posible OCP en la práctica.
3. OCP se aplica sobre puntos de variación conocidos o repetidos — no de forma especulativa.
4. **LSP**: toda subclase debe poder sustituir a su superclase sin alterar la corrección del programa.
5. Señales de violación de LSP: excepciones nuevas, precondiciones más fuertes, postcondiciones más débiles.
6. LSP es el criterio formal que decide si una relación de herencia es segura de usar.



CIERRE

Para la próxima semana

Revisen las jerarquías de herencia de su modelo de diseño: para cada subclase, pregúntense si puede sustituir a su superclase sin lanzar excepciones nuevas ni exigir condiciones adicionales. Si alguna no puede, tráiganla identificada.

La semana 9 cierra la Unidad 4 con los dos principios SOLID que faltan —**Interface Segregation** y **Dependency Inversion**— y un laboratorio de casos de estudio aplicando los cinco principios completos.

